EECS 3311 Course Project - Stage 1

By: Rafat Kabir

Task 1.1

- There are many students that simply don't understand how to study properly for their classes. They struggle to develop lecture notes, memorize important concepts for their classes, create flashcards, etc. There is also another caveat to this. The traditional way of actually developing notes is very tedious and time-consuming. Such as taking handwritten notes, creating flashcards, summarizing, etc. By using an AI we can automate this process, and make it so that the traditional note-taking process can be built upon and enhanced.
- The main users of this application will be any type of student, ranging from elementary students, all the way to even Masters/MBA students.
- The agent is going to have many features. These include the ability to create personalized flashcards for any of your courses given lecture notes, the ability to develop practice tests and quizzes, study reminders, and much more!
- An AI would be appropriate for this problem because many students are different, and require personalized study materials, instead of just 1 fixed set of materials. Instead of having a manually operated system, an AI agent can combine data from various pieces of information, such as previous study sessions, uploaded course materials, etc. An LLM can learn from the student's previous mistakes as well, and develop personalized quiz/test questions based on their weaknesses and strengths.
- I am going to use ChatGPT.
- Our agent is going to be well integrated into the backend of our system. There will be a frontend, a backend, and a database to manage all the previous sessions, lectures notes, and any other data that will be useful to the AI. Whenever the frontend receives some information from the user, we will call our backend RESTful API which will then utilize the AI agent to do work for us using Langchain and RAG for context.

Task 1.2

- **Course Creation**
    - Feature ID: F01
    - This Feature allows users to create and manage courses. Each of these courses are going to act as a hub for a particular course, where it's going to store study materials, generating different kinds of study resources, tracking progress, etc.
    - They will simply click a button in the frontend, something along the lines of "Create Course", where they will then be taken to a form to fill in fields for a course their taking. After submitting this form, the new course will appear.
    - Course name, Course description, Course start date, Course End date.

    - Newly generated Course visible on the user dashboard.
    - Deterministic, since creating a course does not necessarily need an AI model
    - Expected Workflow
        - User presses “Create Course” button
        - User fills in the form and submits the form.
        - User creates course.
    - Missing fields/Invalid fields, User is not authenticated, or at any point the API fails to process.
- **Upload Learning Materials**
    - Feature ID: F02
    - This Feature allows users to upload learning materials to courses, so that the AI can utilize it to develop materials for the student.
    - When user opens the course page, they can select the “upload learning material button”, which will prompt the user to choose one or more files.
    - Input will be the files that the user enters
    - Output will be the new learning materials that are uploaded to the course page
    - It will be hybrid, because uploading materials will only require effort from the user, while processing that material will be from the agent.
    - Expected Workflow
        - User presses “upload learning materials” button
        - Prompts user to upload 1 or more files
        - Files get shown on course page for processing
    - Invalid file type, file exceeds size limit, file is corrupted.
- **Document Summarization**
    - Feature ID: F03
    - This Feature allows users to create summaries for their documents.
    - When user selects the “Summarize document” button under a specific document, they can summarize that document
    - The input would be just clicking the button in this case.
    - Output will be the summarization of said document .
    - It will be AI-based, since the agent will just summarize what's on the document.
    - Expected Workflow
        - User presses “Summarize document" button under certain document
        - Agent summarize document for the user to see
    - Document contains no data that is extractable by the AI, document is too big to let the AI process within the token limits.
- **Concept Extraction**
    - Feature ID: F04

    - This feature allows user to extract important concepts and details from any uploaded learning materials/sections.
    - When user opens course and goes to a particular section, they can press the “Extract Concepts” button, which will then display these concepts in a list in the frontend.
    - The input would be just clicking the Extract Concepts button for a particular section of a course
    - The output would be a list of important concepts
    - I will be AI-based, since it requires the AI to extract all the data and create the list of concepts.
    - Expected Workflow
        - User presses “Extract Concepts” button
        - AI generates all extracted concepts and displays in the frontend
    - The document contains not enough data, model ends up giving duplicate concepts, or the output cannot be properly parsed from the agent.
- **Question Answering Over Materials;**
    - Feature ID: F05
    - This feature allows users to question their study particular for a particular course, and receive proper contextual information
    - User opens course, where they can see the chat interface for the course, and they enter a particular question about the course content.
    - The input would be just be a question entered by the user
    - The output would be the answer to that question/prompt from that user by the AI
    - I will be AI-based, since it requires the AI to process the question and provide an answer to it based on the contextual data.
    - Expected Workflow
        - User goes into course and enters question in the chatbox
        - Agent generates answer to user’s question
    - Agent doesn’t have enough context, and ends up hallucinating.
- **Personalized Study-Plan Generation**
    - Feature ID: F06
    - This feature allows users to create personalized and customized study plan that fit their own schedule for that particular course. This study-plan generation takes into account many things, such as the course material, quiz dates and exam dates, the kind of study time that the user has, etc.
    - User simply selects the “Generate study-plan” button from the course dashboard
    - The input would be a variety of data that contribute to the study-plan generation, including course material, all exam dates, quiz/exam results, and available study time

    - The output would a personalized schedule to help the student study, taking into account time-allocation, and important topics to learn for the user
    - It would be a hybrid effort. An LLM would would handle things like priorities, learning activities, and learning content based on the student, and time allocation and scheduling would be handled by logic.
    - Expected Workflow
        - User selects course that they want to generate a study plan for
        - User presses “Generate study-plan” button
        - System gets student’s own data and habits, time-allocation, and uses that data along with learning material to generate a personalized study plan
    - If user doesn’t enter the proper data, or data is invalid. Available time is not sufficient enough to provide a proper study-plan.
- **Flashcard Generation**
    - Feature ID: F07
    - This feature allows users to create flashcards for a particular courses content/learning material
    - User simply selects the “Generate Flashcards” button in a course. The user can specify the limit to the number of flashcards they want, and the particular sections of a course that they want it for
    - The input would be a form that consists of the limit of the number of flashcards, and the sections they want the flashcards to be based off of
    - The output would a set of flashcards with a question and an answer when the user clicks it.
    - It would be AI-based, as the flashcard content would be handled by the agent.
    - Expected Workflow
        - User selects a course they want to generate flashcards for
        - User presses “Generate Flashcards” button
        - User entered flashcard limit and sections they want it to be based off of
        - System generates all flashcards
    - If the user doesn’t enter the proper data, or data is invalid. Data isn’t sufficient enough to provide a sufficient set of flashcards
- **Quiz Generation**
    - Feature ID: F08
    - This feature allows users to create quizzes based on course/section content.
    - User simply selects the “Generate quiz” button for the whole course or a course section, and they can choose the topics they want to cover, number of questions, and difficulty of quiz

    - The input would be a form that consists of the topics they want to cover, numbers of questions, and difficulty of quiz. There's also some optional fields that the user can enter for the preferred types of questions.
    - The output would be quiz generated by the agent that consists of questions with the content
    - It would be AI-based. The LLM would be responsible for generated all the quiz questions and material depending on what the content it processed is.
    - Expected Workflow
        - User selects a course they want to generate a quiz for, or they go to a particular section of the course to make it granular.
        - User presses “Generate Quiz” button
        - User fills in form and submits
        - System generates all the questions for quiz
    - If the user doesn’t enter the proper data, or data is invalid. Data isn’t sufficient enough to provide a sufficient set of questions
- **Automatic Quiz Grading**
    - Feature ID: F09
    - This feature allows users to get feedback from when they complete their quiz generated by the AI.
    - User presses “SubmitQuiz”, and the agent starts to analyze the user’s answers, and give proper feedback accordingly.
    - Input would be the submitted quiz responses, the questions as well as the expected answers, grading criteria, and point allocation for each question.
    - Output would be a total score, and feedback for identifying exactly which areas the student needs to improve on
    - It would be a hybrid effort. Simple questions like MCQ or true or false questions would be graded by the system, while the agent handles more complex questions and giving more complicated feedback.
    - Expected Workflow
        - User presses “Submit Quiz”
        - AI and system analyze the quiz, grade it, and give feedback.
    - If some questions were unanswered, it could lead to an exception, rubric or grading criteria is missing, AI grading system fails.
- **Calender**
    - Feature ID: F10
    - This feature allows track important course dates, and relevant information, that other parts of the application can utilize, and vice versa. It integrates well with the rest of the application
    - Users can access this feature in the general dashboard, where there will be a section for the calendar, where they’ll be met with a calendar where they can

view anything course related such as deadlines, quizzes, study sessions, assignments.
- Input will be various bits and pieces of user's already established data, or new data the user can entered, including course deadlines, exam dates, assignment dates, planned study sessions, etc.
- Output will be a frontend calendar that will display all the data that the user has entered into a calendar.
- This calendar will be hybrid. The AI will determine timing of study sessions, learning activities, and mock quizzes, while date validation and event creation will be largely deterministic.

Task 2.1

UML Diagram is in Github Repository under diagrams/UML diagrams

1. **Strategy Pattern**
    - Quiz grading can be performed in 2 different ways; it can either be performed deterministically or via AI. Putting both of these algorithms into 1 class would create complex conditional logic
    - QuizGradingService, GradingStrategy, DeterministicGradingStrategy, and AIGradingStrategy.
    - QuizGradingService is the main context, and properly delegates grading to the strategy that's going to be used. GradingStrategy is the grading interface, DeterministicGradingStrategy is the class that uses deterministic grading, and AIGradingStrategy is the class that uses AI based grading
    - Application has more than one grading algorithm, and both algorithms achieve the same general purpose, so strategy would be the most appropriate.
    - The class QuizGradingService would need a large/complex set of conditional logic, like a switch statement or multiple if else statements.
2. **Observer Pattern**
    - The calendar depends on many parts of this application, including course dates, quizzes, and study plan sessions that may need to appear on the calendar if there were changes made. All these classes shouldn't contain calendar logic when a piece of data changes
    - ScheduleSubject, ScheduleObserver, Course, StudyPlan, Quiz, and Calendar.
    - ScheduleSubject is the subject, and defines operations like attach(), detach(), and notifyObservers(). Course, StudyPlan, and Quiz mainly act as concrete subjects, because there's a lot of data related to schedules that can change in them. ScheduleObserver is the observer, and has the update() operation. Calendar is the concrete observer and is the main class that updates the events within it whenever it receives any notification.

- The Calendar is dependent on many other classes in the application. And it needs to react to these changes to those other classes accordingly. The observer and subject relationship is designed for this.
- The Course, StudyPlan, and Quiz classes would constantly need to directly call the Calendar class whenever something changes. This is redundant, and if another component got added later and also needed those same notifications, it would again need to call the Calendar class, and all the previously mentioned classes would need to be modified again.

3. **Facade Pattern**
    - Application contains many different components that require AI. Without a centralized Facade, there are many classes, such as CourseChat that wouldn't know which AI agent to use.
    - StudyAssistantFacade, ContentAgent, PlanningAgent, and AssessmentAgent.
    - StudyAssistantFacade is the main Facade. It contains many different operations for AI including summarize(), generateStudyPlan(), and generateQuiz(). The rest of the classes are part of the AI subsystem that form of the Facade.
    - The application needs a way to request an AI agent's functionality. Because of this, it shouldn't need to directly understand any of the internal components and workings of the AI
    - There would be many classes in the application that would depend directly on many different agents. There would need to be modifications to those classes as well if some of the responsibilities were moved between different agents, or if the subsystem was arranged differently. The facade allows us to hide that complexity from the rest of the system.
4. **Singleton Pattern**
    - There are many different AI agents that require access to the actual LLM that processes everything. We would need to create a separate AIEngine for every single agent for all of these different classes without a singleton, which would by redundant.
    - AIEngine, ContentAgent, PlanningAgent, AssessmentAgent, and AIGradingStrategy
    - The AIEngine class is the singleton, while the agent classes utilize the single AIEngine instance to carry out all of the LLM requests.
    - This pattern is appropriate because the class AIAgent would be shared entity that allows the rest of the application to seamlessly access an AI service.
    - Every single AI component would need to call its own AIService, which would be quite tedious
5. **Factory Pattern**

- There are many kinds of study resources that the application can generate. However, client classes (like User) shouldn't need to be responsible for knowing which resource classes need to be instantiated.
- StudyResourceFactory, StudyResource, Summary, ConceptSet, StudyPlan, FlashcardDeck, and Quiz
- StudyResourceFactory is the main factory class that decides which concrete class to instantiate. StudyResource is the abstract class for all the concrete factory classes. The rest are the concrete products that the factory class decides on.
- There are many study resource objects that are conceptually very similar, and belong to the same StudyResource class, but need different classes. If we centralize this whole creation process, then the clients become independent from the different recourse classes
- All the creation logic would be scattered throughout the application, and adding in another study resource might require modifying many different classes as well.

Task 2.2

Use-case diagram is in the Github repository under diagrams/Use-case diagrams

1. **UC-01**
    - **Use Case ID:** UC-01
    - **Use Case Name:** Manage Courses
    - **Actor(s):** Student
    - Goal is to allow the student to create and manage different courses depending on their courseload
    - Preconditions are that the student is authenticated, and the application is readily available.
    - The trigger would be that the student selects option to create/manage a course
    - The main success scenario would involve the student creating the course → system displaying the course pressing course creation button→ student entering the course name, description, among other fields → student submits form → course created
    - Possible exception could be that fields are missing or student is not authenticated
    - Postcondition would be a valid course in the system, and the course is available on the course dashboard
    - F01 - Course Creation

2. **UC-02**
    - **Use Case ID:** UC-02
      Use Case Name: Upload Learning Materials
      Actor(s): Student
    - Goal is to allow the student to upload learning materials to a course so that it can be used by the LLM properly for study resources.
    - Preconditions are that the student is authenticated, and the course actually exists in the system
    - The trigger would be that the student selects “Upload Learning Material” button from the course page.
    - The main success scenario would involve the student opening a course → the student selects Upload Learning Materials – > The system prompts the student to select one or more files → The student selects the desired files → The system validates the uploaded files → The system stores the files and associates them with the selected course → The learning materials appear on the course page.
    - Exceptions involve unsupported file types, exceeding the file size limit.
    - Postcondition would be the creation of valid learning materials inside a particular course, and the materials being available for other study materials and resources.
    - F02 — Upload Learning Materials
    -
3. **UC-03**
    - **Use Case ID: UC-03**
    - **Use Case Name: Summarize Document**
    - **Actor(s): Student, AI/LLM Service (ChatGPT)**
    - Generate a small summary of the learning material
    - Preconditions are that the course exists, the learning materials are uploaded, the document has proper extractable content, and the AI is readily available
    - Trigger would be that the student selects “Summarize Document” button for a particular document
    - The main success scenario would involve the student selecting an uploaded document → the student selects Summarize Document → the system retrieves the content of the document → the system sends the relevant content to the AI service → the AI service analyzes and summarizes the document → the system receives the generated summary → the summary is displayed to the student → the summary is stored as a study resource for the course

- Possible exceptions might be that there is no valid content, or the document exceeds the limits from the AI
- Postcondition would be a summary made by the AI from a document
- F03 — Document Summarization

4. **UC-04**
   - **Use Case ID:** UC-04
   - **Use Case Name:** Extract Concepts
   - **Actor(s):** Student, AI/LLM Service (ChatGPT)
   - The goal is to identify different important concepts and details from course materials.
   - Preconditions would be the selected course actually existing, learning materials being available, and the AI service also being available.
   - The trigger would be the student selecting Extract Concepts for either a document or a course section
   - The main success scenario would be: student selects the relevant learning material → the student selects Extract Concepts → the system retrieves the relevant content → the content is provided to the AI service → the AI analyzes the material and identifies important concepts → the system processes the generated concepts → the concepts are displayed to the student
   - A common exception could be that the material contains not enough information, or the AI generates duplicate concepts. The AI's output could also just not be parsed correctly
   - Postcondition would be a set of important concepts for the course
   - F04 — Concept Extraction
5. **UC-05**
   - **Use Case ID:** UC-05
   - **Use Case Name:** Ask Questions About Course Materials
   - **Actor(s):** Student, AI/LLM Service (ChatGPT)
   - The goal is to allow students to ask questions and receive proper feedback about course material
   - Preconditions would be the course actually existing, learning materials being available, student being able to access the chat, and the AI service being available
   - The trigger would be the student entering and submitting a question through the course's chat window
   - The main success scenario would be: student opens a course → the student opens the course chat → the student enters a question → the system retrieves relevant information from the course materials → the question and relevant context are provided to the AI service → the AI service generates an

answer → the system displays the answer to the student → the question and response are stored in the course chat history
    - A common exception could be that the course doesn't contain enough data, the data cannot be retrieved, or the AI starts hallucinating.
    - Postcondition would be the student receiving a response, and the response and interaction being saved
    - F05 — Question Answering Over Materials
6. **UC-06**
    - **Use Case ID:** UC-06
    - **Use Case Name:** Generate Personalized Study Plan
    - **Actor(s):** Student, AI/LLM Service (ChatGPT)
    - The goal is to allow students to generate a personalized study plan depending on the course material, study time, study habits, etc
    - Preconditions would be the course existing, relevant materials being available, study-profile being available, scheduling info being available, and the AI service being available.
    - The trigger would be the student selecting "Generate study plan" for a course
    - The main success scenario would be: student opens the desired course → the student selects Generate Study Plan → the system retrieves the student's study profile, including availability, preferences, and study habits → the system retrieves relevant course material, important dates, and previous assessment information → the AI service analyzes the course content and determines priorities, important topics, and appropriate learning activities → the system applies scheduling logic to the student's available study time → the system creates individual study sessions → the system creates the personalized study plan → the completed study plan is displayed to the student
    - A common exception could be that the study-profile is missing, study time is simply insufficient, course material isn't sufficient either, or the required course dates are invalid.
    - Postcondition would be the personalized study plan being created, study sessions being associated with the course at hand, and the generated sessions reflecting in the calendar
    - F06 — Personalized Study-Plan Generation
7. **UC-07**
    - **Use Case ID:** UC-07
    - **Use Case Name:** Generate Flashcards
    - **Actor(s):** Student, AI/LLM Service (ChatGPT)
    - The goal would be to generate a deck of flashcards according to the course material

- Preconditions would be the course existing, the relevant materials are available, and the AI service is readily available.
- The trigger would be the student selecting “Generate Flashcards” for a course
- The main success scenario would be: student opens the desired course → the student selects Generate Study Plan → the system retrieves the student’s study profile, including availability, preferences, and study habits → the system retrieves relevant course material, important dates, and previous assessment information → the AI service analyzes the course content and determines priorities, important topics, and appropriate learning activities → the system applies scheduling logic to the student’s available study time → the system creates individual study sessions → the system creates the personalized study plan → the completed study plan is displayed to the student
- A common exception could be that the flashcard limit is invalid, the course section selected is invalid, insufficient learning material, or the AI service simply fails to generate proper flashcards
- Postcondition would be the flashcard deck of cards being generated, and the material being associated with the course
- F07 — Flashcard Generation

8. **UC-08**
   - **Use Case ID:** UC-08
   - **Use Case Name:** Generate Quiz
   - **Actor(s):** Student, AI/LLM Service (ChatGPT)
   - The goal would be to generate a mock-quiz of questions related to the course content
   - Preconditions would be the course existing, the relevant materials are available, and the AI service is readily available.
   - The trigger would be the student selecting “Generate quiz” for a course/course section
   - The main success scenario would be: student selects a course or course section → the student selects Generate Quiz → the system displays a quiz-generation form → the student selects the topics to cover → the student specifies the number of questions → the student selects the difficulty level → the student may specify preferred question types → the system validates the entered information → the relevant course material is provided to the AI service → the AI generates the requested questions → the system creates the quiz → the quiz is displayed to the student
   - A common exception could be that the number of questions is invalid, the selected topics being invalid, insufficient course material, or the AI can’t generate questions related to the course material

- Postcondition would be the quiz being generated, and the student being able to attempt it and submit it
- F08 — Quiz Generation

9. **UC-09**
   - **Use Case ID:** UC-09
   - **Use Case Name:** Submit Quiz and Receive Feedback
   - **Actor(s):** Student, AI/LLM Service (ChatGPT) when AI-assisted grading is required
   - The goal would be to submit the quiz from UC-08, and receive proper feedback from the AI
   - Preconditions would be a quiz existing, and the user completed/attempting the quiz, and the grading criteria being available
   - The trigger would be the student selecting “Submit Quiz” for a quiz
   - The main success scenario would be: student completes the quiz → the student submits the quiz → the system receives the student’s responses → the system determines the appropriate grading method for each question → objective questions, such as multiple-choice or true/false questions, are graded deterministically → complex or open-ended questions use AI-assisted grading when necessary → the system combines the grading results → the total quiz score is calculated → weak areas are identified → feedback is generated → the score and feedback are displayed to the student
   - A common exception could be that one or more questions are left unanswered, the grading criteria being unavailable, the grading isn’t appropriate and not on par, and the response cannot be processed.
   - Postcondition would be the grade result being created, the student receiving proper feedback and a score, and weak areas being identified for future study sessions/resources.
   - F09 — Automatic Quiz Grading
10. **UC-10**
   - **Use Case ID:** UC-10
   - **Use Case Name:** View / Manage Calendar
   - **Actor(s):** Student
   - The goal would be to view and manage a calendar with important academic dates, study events, deadlines, etc.
   - Preconditions would be the student being authenticated, and the student’s calendar being readily available on the dashboard
   - The trigger would be the student opening the calendar from the main application dashboard
   - The main success scenario would be: student opens the calendar → the system retrieves the student’s scheduled information → course deadlines, exam dates, quiz dates, assignment dates, and planned study sessions are

displayed → the student reviews the scheduled events → the student may add a new calendar event → the student may remove an existing event → the system validates any changes → the updated calendar is displayed

- A common exception could be that an invalid time/date could be entered while putting in information in the calendar, event information is missing, or calendar information from the subjects not being able to be retrieved
- Postcondition would be the calendar containing all the relevant data, and the any changes related to the calendar elsewhere updating the calendar accordingly.
- F10 — Calendar

11. UC-11

- **Use Case ID:** UC-11
- **Use Case Name:** Maintain Study Profile
- **Actor(s):** Student
- The goal would be to allow the student/user to maintain proper information regarding their study availability, their preferences, and their habits so that the study-plan can use this information
- Preconditions would be the student being authenticated, and the student's study profile existing in the same.
- The trigger would be the student choosing to update their study profile
- The main success scenario would be: student opens their study profile → the system displays the student's current study-profile information → the student updates their available study time, study preferences, or study habits → the student submits the changes → the system validates the entered information → the system saves the updated study profile → the new profile information becomes available for personalized study-plan generation
- A common exception could be that the entered information when the student is updating their profile is simply invalid, the required info missing, or the system fails to update their data.
- Postcondition would be the student having up-to-date information on their profile
- F06 — Personalized Study-Plan Generation

Task 2.3

SD01-SD06 Sequence Diagrams are all in the Github Repository under diagrams/Sequence diagrams

Task 3

F01: Course Creation
Type: Deterministic
Related Use Case: UC-01 Manage Courses
Classes: User, Course
Key Methods: User.createCourse(), User.viewCourses()
Sequence Diagram: SD01
Design Pattern(s): None / Core Domain

F02: Upload Learning Materials
Type: Hybrid
Related Use Case: UC-02 Upload Learning Materials
Classes: Course, LearningMaterial, StudyAssistantFacade, ContentAgent, AIEngine
Key Methods: Course.addMaterial(), LearningMaterial.getContent(), StudyAssistantFacade.processMaterial(), ContentAgent.processMaterial(), AIEngine.generate()
Sequence Diagram: SD01
Design Pattern(s): Facade, Singleton

F03: Document Summarization
Type: AI-Based
Related Use Case: UC-03 Summarize Document
Classes: LearningMaterial, StudyAssistantFacade, ContentAgent, AIEngine, StudyResourceFactory, Summary
Key Methods: LearningMaterial.getContent(), StudyAssistantFacade.summarize(), ContentAgent.summarize(), AIEngine.getInstance(), AIEngine.generate(), StudyResourceFactory.createResource()
Sequence Diagram: SD02
Design Pattern(s): Facade, Singleton, Factory

F04: Concept Extraction
Type: AI-Based
Related Use Case: UC-04 Extract Concepts

Classes: LearningMaterial, StudyAssistantFacade, ContentAgent, AIEngine, StudyResourceFactory, ConceptSet
Key Methods: LearningMaterial.getContent(), StudyAssistantFacade.extractConcepts(), ContentAgent.extractConcepts(), AIEngine.generate(), StudyResourceFactory.createResource()
Sequence Diagram: SD02
Design Pattern(s): Facade, Singleton, Factory

F05: Question Answering Over Materials
Type: AI-Based
Related Use Case: UC-05 Ask Questions About Course Materials
Classes: Course, CourseChat, StudyAssistantFacade, ContentAgent, AIEngine
Key Methods: Course.askQuestion(), CourseChat.askQuestion(), CourseChat.viewHistory(), StudyAssistantFacade.answerQuestion(), ContentAgent.answerQuestion(), AIEngine.generate()
Sequence Diagram: SD02
Design Pattern(s): Facade, Singleton

F06: Personalized Study-Plan Generation
Type: Hybrid
Related Use Case: UC-06 Generate Personalized Study Plan; UC-11 Maintain Study Profile
Classes: Course, StudyProfile, StudyAssistantFacade, PlanningAgent, AIEngine, StudyResourceFactory, StudyPlan, StudySession, Calendar
Key Methods: StudyAssistantFacade.generateStudyPlan(), PlanningAgent.generateStudyPlan(), AIEngine.generate(), StudyResourceFactory.createResource(), StudyPlan.getSessions(), ScheduleSubject.notifyObservers(), Calendar.update()
Sequence Diagram: SD03, SD06
Design Pattern(s): Facade, Singleton, Factory, Observer

F07: Flashcard Generation
Type: AI-Based
Related Use Case: UC-07 Generate Flashcards
Classes: Course, StudyAssistantFacade, AssessmentAgent, AIEngine, StudyResourceFactory, FlashcardDeck
Key Methods: StudyAssistantFacade.generateFlashcards(), AssessmentAgent.generateFlashcards(), AIEngine.generate(), StudyResourceFactory.createResource(), FlashcardDeck.viewCards()
Sequence Diagram: SD04
Design Pattern(s): Facade, Singleton, Factory

F08: Quiz Generation
Type: AI-Based
Related Use Case: UC-08 Generate Quiz
Classes: Course, StudyAssistantFacade, AssessmentAgent, AIEngine, StudyResourceFactory, Quiz
Key Methods: StudyAssistantFacade.generateQuiz(), AssessmentAgent.generateQuiz(), AIEngine.generate(), StudyResourceFactory.createResource(), Quiz.startQuiz()
Sequence Diagram: SD04
Design Pattern(s): Facade, Singleton, Factory

F09: Automatic Quiz Grading
Type: Hybrid
Related Use Case: UC-09 Submit Quiz and Receive Feedback
Classes: Quiz, QuizGradingService, GradingStrategy, DeterministicGradingStrategy, AIGradingStrategy, AIEngine, GradeResult
Key Methods: Quiz.submitQuiz(), QuizGradingService.setStrategy(), QuizGradingService.gradeQuiz(), DeterministicGradingStrategy.grade(), AIGradingStrategy.grade(), AIEngine.generate()
Sequence Diagram: SD05
Design Pattern(s): Strategy, Singleton

F10: Calendar
Type: Hybrid
Related Use Case: UC-10 View / Manage Calendar
Classes: Calendar, ScheduleSubject, ScheduleObserver, Course, StudyPlan, Quiz
Key Methods: Calendar.addEvent(), Calendar.removeEvent(), Calendar.viewEvents(), Calendar.update(), ScheduleSubject.attach(), ScheduleSubject.detach(), ScheduleSubject.notifyObservers()
Sequence Diagram: SD06
Design Pattern(s): Observer

Task 4

F01: Course Creation
Related Use Case: UC-01 — Manage Courses
Related Sequence Diagram: SD01 — Create Course and Upload Learning Materials
Classes involved:

- **User:** represents the student and maintains the courses owned by the user.
- **Course:** represents an individual course containing its name, description, start date, end date, materials, and study resources.

Important methods:

- User.createCourse()

- User.viewCourses()

Execution:

When the student selects the option to create a course, the data from the course is collected from the user, and validated. System then calls User.createCourse to create a new Course object, and associate that Course object with the student. And once this course is created, it becomes available in the course dashboard through User.viewCourses(). If any of the fields are missing or invalid, then the system will return an error instead. This directly corresponds to the course creation behaviour in F01 and UC-01

F02: Upload Learning Materials
Related Use Case: UC-02 — Upload Learning Materials
Related Sequence Diagram: SD01 — Create Course and Upload Learning Materials
Classes involved:

- **Course:** represents the selected course and stores the learning materials associated with it.
- **LearningMaterial:** represents an uploaded course document or file and provides access to its content.

Important methods:

- Course.addMaterial()
- LearningMaterial.getContent()

Execution:

When the student selects the option to upload learning materials, the system prompts the user to choose either one or more files. These uploaded files are then validated to make sure that the type of the file and size are all acceptable. If all the files are valid, the system creates LearningMaterial objects according to however many files were submitted, and then associates them all with the selected course using Course.addMaterial(). This material can be accessed later by using LearningMaterial.getContent() for access to a variety of different features. If the file type is unsupported or its too large/corrupted, then the system will return an error

F03: Document Summarization
Related Use Case: UC-03 — Summarize Document
Related Sequence Diagram: SD02 — AI Content Operations
Classes involved:

- **LearningMaterial:** contains the document content that will be summarized.
- **StudyAssistantFacade:** provides a simplified interface for accessing the AI subsystem.
- **ContentAgent:** handles AI operations related to course content.
- **AIEngine:** provides shared access to the LLM service.
- **StudyResourceFactory:** creates the resulting study resource.
- **Summary:** represents the generated document summary.

Important methods:

- LearningMaterial.getContent()
- StudyAssistantFacade.summarize()
- ContentAgent.summarize()
- AIEngine.getInstance()
- AIEngine.generate()
- StudyResourceFactory.createResource()

Execution:

When the student selects Summarize Document, the system will retrieve the content of the document using LearningMaterial.getContent(). This request is passed to StudyAssistantFacade.summarize(), which will then push that work to ContentAgent.summarize(). ContentAgent will use AIEngine.getInstance() to get the shared AIEngine and call AIEngine.generate() to send the document's context and summarization request to the AI Service. When the generated summary is then returned, StudyResourceFactory will create a Summary object. This completed summary will then be displayed to the student, and it will be related to the course. If the document doesn't contain any extractable data, or exceeds to AI's processing limits, then the system shall return an error

F04: Concept Extraction

Related Use Case: UC-04 — Extract Concepts

Related Sequence Diagram: SD02 — AI Content Operations

Classes involved:

- **LearningMaterial:** contains the course content from which important concepts are extracted.
- **StudyAssistantFacade:** provides access to the AI subsystem.
- **ContentAgent:** performs AI-based concept extraction.
- **AIEngine:** communicates with the LLM service.
- **StudyResourceFactory:** creates the resulting study resource.
- **ConceptSet:** stores the extracted concepts.

Important methods:

- LearningMaterial.getContent()
- StudyAssistantFacade.extractConcepts()
- ContentAgent.extractConcepts()
- AIEngine.generate()
- StudyResourceFactory.createResource()

Execution:

When the student selects Extract Concepts, the system retrieves the relevant content from the selected LearningMaterial. The system calls StudyAssistantFacade.extractConcepts(), which delegates the request to ContentAgent.extractConcepts(). ContentAgent then uses AIEngine.generate() to send the content to the AI service and request a list of important

concepts. Once the result is returned, StudyResourceFactory creates a ConceptSet object containing the generated concepts. The concepts are then displayed to the student. If the material contains insufficient information, duplicate concepts are generated, or the AI output cannot be processed correctly, the system returns an error.

F05: Question Answering Over Materials
Related Use Case: UC-05 — Ask Questions About Course Materials
Related Sequence Diagram: SD02 — AI Content Operations
Classes involved:

- **Course:** represents the course whose materials provide the context for the question.
- **CourseChat:** manages questions and the course chat history.
- **StudyAssistantFacade:** provides access to the AI question-answering functionality.
- **ContentAgent:** generates contextual answers using course material.
- **AIEngine:** communicates with the external AI service.

Important methods:

- Course.askQuestion()
- CourseChat.askQuestion()
- CourseChat.viewHistory()
- StudyAssistantFacade.answerQuestion()ContentAgent.answerQuestion()
- AIEngine.generate()

Execution:
When the user submits a question in the course's chatbot, CourseChat.askQuestion() sends a request to studyAssistantFacade.answerQuestion(). The facade sends the request to ContentAgent.answerQuestion(), which then uses the course material as context. ContentAgent then calls AIEngine.generate() to send the question and the context of the course to the AI service. The generated answer then is returned using the Facade to Coursechat and is properly displayed to the user. If there isn't enough course data available, then the application will return an error indicating that there isn't enough context available.

F06: Personalized Study-Plan Generation
Related Use Case: UC-06 — Generate Personalized Study Plan
Related Sequence Diagram: SD03 — Generate Personalized Study Plan
Classes involved:

- **Course:** provides the relevant course material and course information.
- **StudyProfile:** stores the student's available study time, preferences, and habits.
- **StudyAssistantFacade:** provides access to study-plan generation.
- **PlanningAgent:** generates study priorities and learning activities.
- **AIEngine:** communicates with the external AI service.

- **StudyResourceFactory:** creates the resulting StudyPlan.
- **StudyPlan:** represents the complete personalized study plan.
- **StudySession:** represents an individual study session within the plan.
- **Calendar:** displays and synchronizes generated study sessions.

Important methods:

- StudyProfile.updateAvailability()
- StudyProfile.updatePreferences()
- StudyAssistantFacade.generateStudyPlan()
- PlanningAgent.generateStudyPlan()
- AIEngine.generate()
- StudyResourceFactory.createResource()
- StudyPlan.getSessions()
- ScheduleSubject.notifyObservers()
- Calendar.update()

Execution:

Whenever the user selects Generate Study Plan button, the system will receive the user's StudyProfile, course material, important dates, etc. StudyAssistantFacade.generateStudyPlan() puts the AI-related part of the request into PlanningAgent.generateStudyPlan(). This PlanningAgent will then use AIEngine.generate() to observe important topics, priorities, and learning activities that are appropriate. The application applies the scheduling logic afterwards based on whatever the student's available study availability is, and will create a StudyPlan containing StudyPlan objects. The generated sessions are then in sync with the Calender using notifyObservers() and Calendar.invite(). In case the study profile is incomplete, or the course material is insufficient, or there simply isn't enough available study time, the application will throw an error.

F07: Flashcard Generation

Related Use Case: UC-07 — Generate Flashcard

Related Sequence Diagram: SD04 — Generate Flashcards and Quiz

Classes involved:

- **Course:** provides the course material used to generate flashcards.
- **StudyAssistantFacade:** provides access to flashcard-generation functionality.
- **AssessmentAgent:** generates the flashcard content.
- **AIEngine:** communicates with the external AI service.
- **StudyResourceFactory:** creates the flashcard resource.
- **FlashcardDeck:** represents the generated set of flashcards.

Important methods:

- StudyAssistantFacade.generateFlashcards()
- AssessmentAgent.generateFlashcards()
- AIEngine.generate()
- StudyResourceFactory.createResource()
- FlashcardDeck.viewCards()

Execution:

When the user presses the Generate Flashcards button, the system will collect various pieces of information such as the number of flashcards limit, and the course sections that it should create it for. When the request is validated, StudyAssistantFacade.generateFlashcards() puts the request to AssessmentAgent.generateFlashcards(). The AssessmentAgent calls AIEngine.generate() to generate the flashcards actual content based on the course selected. StudyResourceFactory will create a FlashcardDeck containing all the generated cards. The student can then view the cards using FlashcardDeck.viewCards(). If in case the flashcard limit invalid, the sections are invalid, the material is simply insufficient, or the AI cannot generate the appropriate flashcards, the system will return an error to the user.

F08: Quiz Generation

Related Use Case: UC-08 — Generate Quiz

Related Sequence Diagram: SD04 — Generate Flashcards and Quiz

Classes involved:

Course: provides the course material used to generate the quiz.

- **StudyAssistantFacade:** provides access to quiz-generation functionality.
- **AssessmentAgent:** generates quiz questions using the course content.
- **AIEngine:** communicates with the external AI service.
- **StudyResourceFactory:** creates the resulting quiz resource.
- **Quiz:** represents the generated quiz.

Important methods:

- StudyAssistantFacade.generateQuiz()
- AssessmentAgent.generateQuiz()
- AIEngine.generate()
- StudyResourceFactory.createResource()
- Quiz.startQuiz()

Execution:

Whenever the student selects “Generate Quiz”, the application will ask the user to provide all the topics, the number of questions they want, the difficulty level, etc. After all this information is processed and validated, StudyAssistantFacade.generateQuiz() puts the request to AssessmentAgent.generateQuiz(). AssessmentAgent uses AIEngine.generate() to generate questions based on the course content and the form that user filled. StudyResourceFactory creates a quiz object afterwards containing all the generated

questions. The quiz thats completed is then available to the user via Quiz.startQuiz(). If the number of questions is invalid, the topics are invalid, or there isn't enough material, then the system returns an error.

F09: Automatic Quiz Grading
Related Use Case: UC-09 — Submit Quiz and Receive Feedback
Related Sequence Diagram: SD05 — Submit Quiz and Hybrid Grading
Classes involved:

- **Quiz:** contains the quiz and the student's submitted responses.
- **QuizGradingService:** coordinates the grading process and chooses the appropriate grading strategy.
- **GradingStrategy:** defines the common interface used by the grading strategies.
- **DeterministicGradingStrategy:** grades objective questions such as multiple-choice and true/false questions.
- **AIGradingStrategy:** grades complex or open-ended responses.
- **AIEngine:** provides AI functionality for AI-assisted grading.
- **GradeResult:** stores the student's score, feedback, and identified weak areas.

Important methods:

- Quiz.submitQuiz()
- QuizGradingService.setStrategy()
- QuizGradingService.gradeQuiz()
- DeterministicGradingStrategy.grade()
- AIGradingStrategy.grade()
- AIEngine.generate()

Execution:
When the student finishes the quiz and selects Submit Quiz, Quiz.submitQuiz() will send this completed version of the quiz to QuizGradingService. The grading service will then determine the grading strategy for each and every question. Questions that have an objective criteria like MCQ or T/F will get graded using DeterministicGradingStrategy.grade, while more complex questions are going to be graded using AIGradingStrategy.grade(), which will use AIEngine.generate(). These results are combined into a GradeResult, which will contain the student's score, and proper feedback identifying their weak areas. The GradeResult will then return and be displayed to the student. If any of the answers are missing, grading criteria is unavailable, or the responses can't be processed for whatever reason, the system will return an error.

F10: Calendar
Related Use Case: UC-10 — View / Manage Calendar
Related Sequence Diagram: SD06 — Calendar and Study Profile
Classes involved:

- **Calendar:** stores and displays the student's academic and study-related events.

- **ScheduleSubject:** defines the operations used by schedule-related classes to notify observers.
- **ScheduleObserver:** defines the update behavior for objects that observe schedule changes.
- **Course:** may contain important dates that affect the calendar.
- **StudyPlan:** contains generated study sessions that need to appear on the calendar.
- **Quiz:** may contain quiz-related dates that affect the calendar.

Important methods:

- Calendar.addEvent()
- Calendar.removeEvent()
- Calendar.viewEvents()
- Calendar.update()
- ScheduleSubject.attach()
- ScheduleSubject.detach()
- ScheduleSubject.notifyObservers()

Execution:

Whenever the student opens the calendar, Calendar.viewEvents() retrieves the academic events that are current, and then displays them to the student. The student can also manually create events in the calendar using Calendar.addEvent(), or remove these events using Calendar.removeEvent(). As we discussed previously the Calendar is an observer of many subjects, including Course, StudyPlan, and Quiz. When schedule related data changes in any one of those subjects, the changes are reflected into Calendar using ScheduleSubject.notifyObservers(), which then causes Calendar.update() to be called and sync the calendar with up to date information. In case invalid is entered or received, event info is missing, or scheduling information cannot be retrieved, the system will return an error